

**19 JULY**

**2024**

# The morning it stopped

## Airports

Departure boards dead. Check-in by hand. Fleets grounded.

## Hospitals

Back to paper records mid-shift.

## Banks

Payment terminals offline. Branches closed.

## Broadcast + emergency

Studios dark. Dispatch systems down.

~8.5M

Windows machines dead  
the same morning

*Microsoft estimate*

# So what happened?

Shout it out.

*[ Do not click until three people have guessed ]*

# Not a hacker

Not a breach.

Not a state actor.

Not really even a bug in the product.

**A configuration update, pushed to every customer at once.**

● No staged rollout.

● No canary group.

● No way to stop it once it started.

**It was not a coding failure. It was a shipping failure.**

# And closer to home

## TODO - LOCAL INCIDENT

One Pakistani outage this room lived through.

CrowdStrike proves it happens. A local incident proves it happens here, to systems built by people like them - which is the version that changes behaviour.

*Only present a figure you can defend out loud. If you cannot source it, cut this slide.*

# DevOps, Cloud, SRE & Platform Engineering

Intern Rotation 2026 | tkxel

Two hours today. Six days of keyboards after that.

# Adeel Arshad

Head of DevOps & Cloud, tkxel

**20+**

years across full-stack, DevOps, SRE, cloud architecture, and CTO/CEO roles

**CNCF**

Kubestronaut - all five Kubernetes certifications. A few thousand worldwide.

**30**

engineers in the DevOps practice I lead, from associate to architect

I also build open-source cloud tooling - observability, cost intelligence, security posture. Ask me about any of it.

**Ask me anything this week - including the questions you think are too basic. Especially those.**

# Before we start - where are you at?

Hands up. Be honest, nobody is marking this.

☐ Excited - I want to automate something

☐ Still processing the last rotation

☐ Nervous but curious

☐ Here for the tools and the tech

☐ I want to see how it all connects

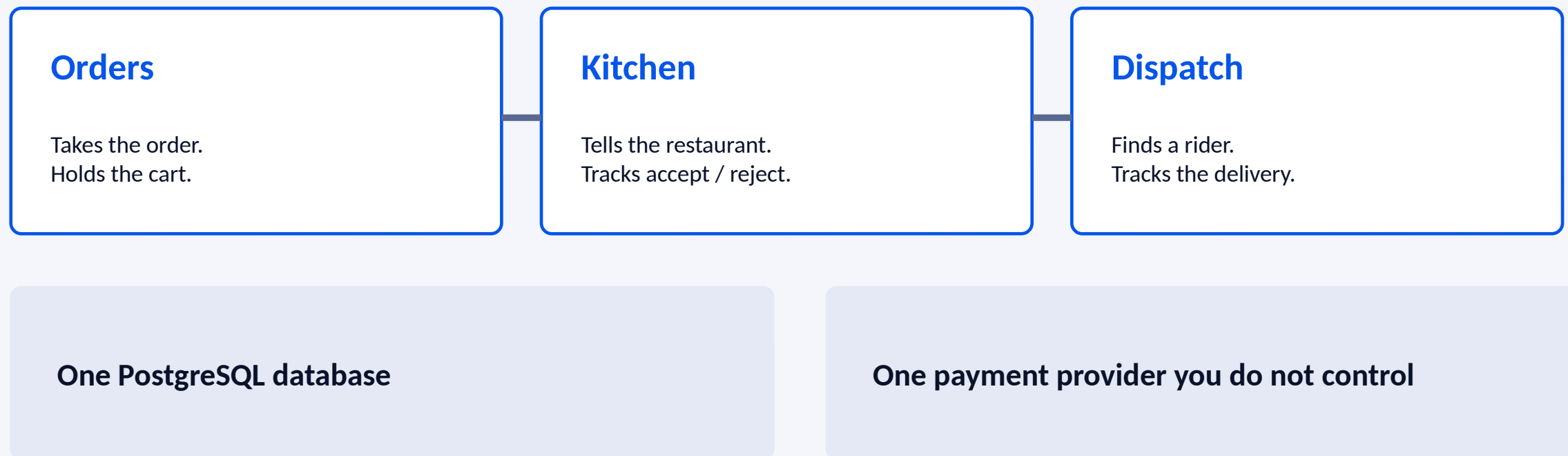
☐ No idea what DevOps actually is

*The last one is the correct answer, by the way. Everyone else is guessing too.*



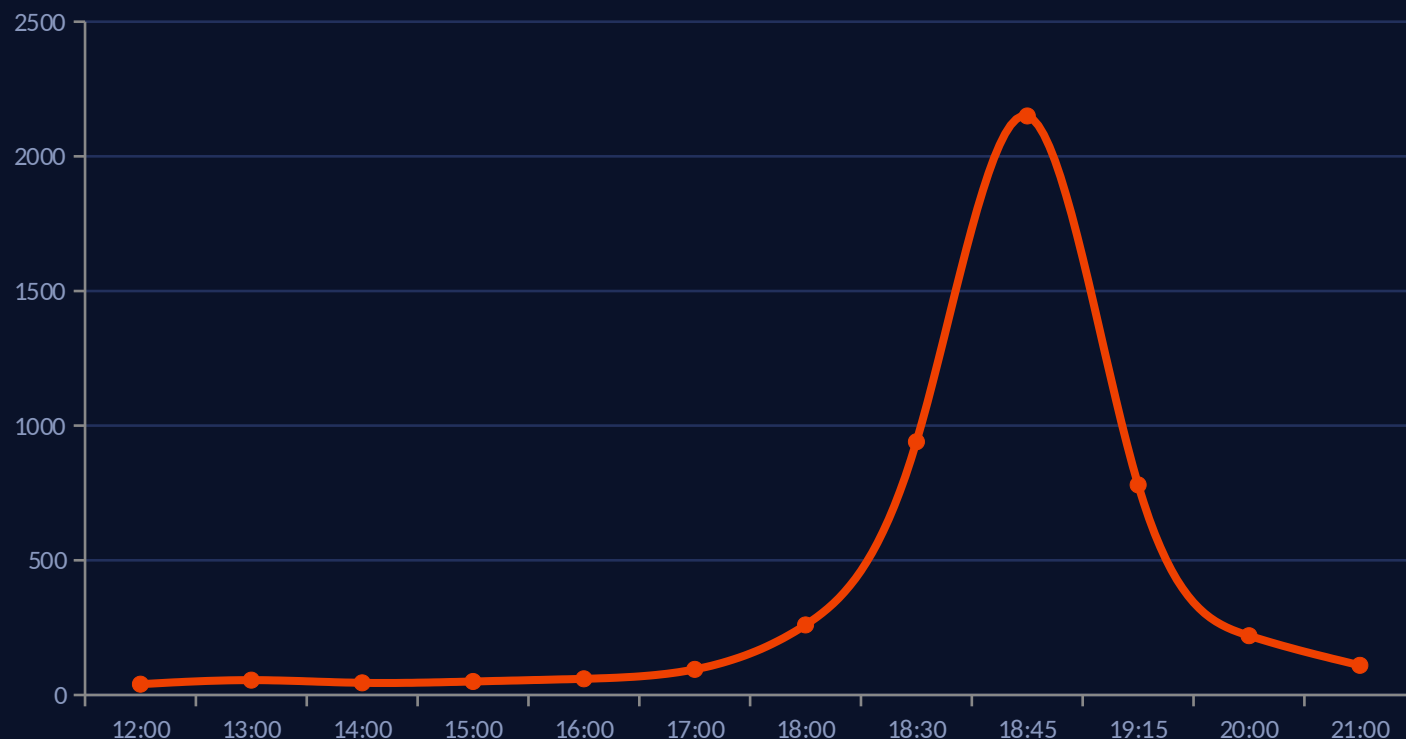
# Meet Daig

Food delivery. Three services, one database, one payment integration. This is the system you will break and fix all week.



*A daig is the pot you cook in when you are feeding everybody at once. That is the entire engineering problem in one word.*

# Daig has one bad twenty minutes a day



## You know it is coming

Every single day. Same time. This is the easiest capacity problem in the world - and teams still fail it.

## Scaling is slower than the spike

New capacity takes ~90 seconds to come up. The spike arrives in 20.

## The deadline is set by the sun

No "please try again later." There is no later.

# What breaks?

You have seen the system. You have seen the traffic. Two minutes - tell me what goes wrong at 18:45.

*[ Write every answer on the board. Leave it up all session. ]*

Every answer you give maps onto a day this week. We will come back to this board and I will point at your words.

*There are no wrong answers on this slide.*

# Your turn. Break it.

```
docker run --name daig-orders tkxel/daig-orders:broken
```

## It will not work. Find out why.

- You have eight minutes.
- You may use any command, any search, any AI tool.
- You may not ask me for the answer.
- Talk to the person next to you.

*If your Docker is not working, pair with someone whose is. That is also the lesson.*

# Look at what you just did

Forget Docker. This is the pattern underneath it.

1

## Observed

You read the logs instead of guessing.  
The system was telling you what was wrong the whole time.

2

## Hypothesised

It cannot reach its database. Narrow, testable, wrong-able.

3

## Changed config, not code

You did not touch the application.  
You changed how it was configured.

4

## Verified

You proved the fix worked instead of hoping.

**That loop, at scale, with automation around it and real money on the line, is the job.  
Everything this week is that loop, larger.**

# So - DevOps

One team owning a system from the idea to the incident, instead of throwing it over a wall.

## WITHOUT IT

Devs write it, someone else runs it

Releases are events people dread

"Works on my machine"

Nobody owns the 3am pager

Blame arrives before the fix

## WITH IT

One team owns build and run

Releases are boring and frequent

Runs the same everywhere, by design

Whoever built it shares the pager

Fix first, learn second, blame never

*Notice none of that is a tool.*

# Four things, in order

CAMS. Fifteen years old and still the clearest framing anybody has produced.



## Culture

People and how they work together. Comes first because nothing else survives without it.



## Automation

If a human does it twice, a machine should do it the third time.



## Measurement

You cannot improve what you refuse to measure.



## Sharing

The one everyone skips. It is what makes the other three stick.

# Three acts

If you remember one slide from today, make it this one.



## PRE-PRODUCTION

*Before users can touch it*

|       |         |
|-------|---------|
| Plan  | Code    |
| Build | Test    |
| Scan  | Package |



## PRODUCTION

*Getting it to users safely*

|         |        |
|---------|--------|
| Release | Deploy |
| Canary  | Config |
| Secrets | Scale  |



## POST-PRODUCTION

*Living with what you shipped*

|            |          |
|------------|----------|
| Observe    | Alert    |
| On-call    | Incident |
| Postmortem | Cost     |

**Most courses cover Act I, mention Act II, and stop. Act III is where the job actually lives.**



# Pre-production: before anyone can be hurt by it

| WHAT        | WHY IT EXISTS                             | SKIP IT AND                                |
|-------------|-------------------------------------------|--------------------------------------------|
| Plan & code | Version control, review, branching        | Nobody knows who changed what, or why      |
| Build       | One repeatable artifact from source       | "It built on my laptop" is not a release   |
| Test        | Automated checks that can block a release | You find out from customers instead        |
| Scan        | Dependencies, secrets, licences, SBOM     | You ship somebody else's vulnerability     |
| Package     | One immutable artifact, staged rollout    | <b>CrowdStrike. Every machine at once.</b> |

*Days 1 to 4 this week. You will build every row.*

# Production: the dial, not the switch

Shipping to everyone at once is a choice - and almost never the right one.

**1%**

Canary. If it is broken, 99 people out of 100 never notice.

**5%**

Still cheap to undo. Watch your error rate and latency.

**50%**

Now you have real signal. Compare the two halves.

**100%**

Only because the previous three steps told you it was safe.

## Configuration

Same artifact, different settings per environment. Never rebuild to change a value.

## Secrets

Passwords and keys are not configuration and never live in Git.

## Scaling

More copies when the iftar spike arrives. Fewer at 3am, so you can afford it.

# Post-production: the half nobody shows you

You have all written code. None of you has been woken at 3am by code.

## Observe

Logs, metrics, traces, profiles. Four questions, one request.

## Alert

Waking a human is expensive. Alert on symptoms, not causes.

## On-call

Someone is responsible right now. This week, that is a rota, not a hero.

## Incident

Roles, comms, decisions under pressure. Practised, not improvised.

## Postmortem

Blameless. Chase the system, not the person.

## Cost & capacity

Reliability you cannot afford is not reliability.

*Thursday afternoon and Friday. Also the rest of your career.*

# Reliability is a budget, not a virtue

**99%**

**7 hours 18 min**

per month

**99.9%**

**43 minutes**

per month

**99.99%**

**4 minutes 20 sec**

per month

**Daig targets 99.9%. That is 43 minutes a month.  
Tonight's incident: 14 minutes. One bad Thursday just spent a third of the month.**

Spend it all and you stop shipping features until the window resets. That is not a punishment - it is the system telling you to fix things before you add things.

# Break

Eight minutes. We start again on the clock -  
which is a habit worth having this week.



# 18:42

**Iftar in eighteen minutes.**

Daig's order service starts returning 503s.  
The pager goes off.

Four people pick it up. They ask four completely different questions.

# Same incident. Four questions.



## DevOps engineer

*How do we get this working again, now?*

Rolls back the 17:30 deploy. Confirms recovery. Then asks how it got through the pipeline.



## SRE

*How much failure can we afford before we stop shipping?*

Checks the error budget: 60% spent, 11 days left in the window. Calls a change freeze.



## Platform engineer

*Why was it even possible to deploy at 17:30 on a Thursday?*

Ships a deployment-window guardrail into the golden path. Now no team can do this again.



## Cloud / FinOps

*We survived by tripling the instances. What did that cost?*

Prices the spike. Models pre-provisioning against the monthly budget.

You will do all four this week. You will specialise later. You do not have to pick today.

# Four words worth stealing

If you use these correctly in an interview, the interviewer will assume you have done the work.

## SLI

*Service Level Indicator*

The thing you actually measure. "99.2% of order requests succeeded."

## SLO

*Service Level Objective*

The target you promise. "99.9% of order requests will succeed."

## Error budget

*SLO minus reality*

How much failure you are allowed. Spend it deliberately.

## Golden path

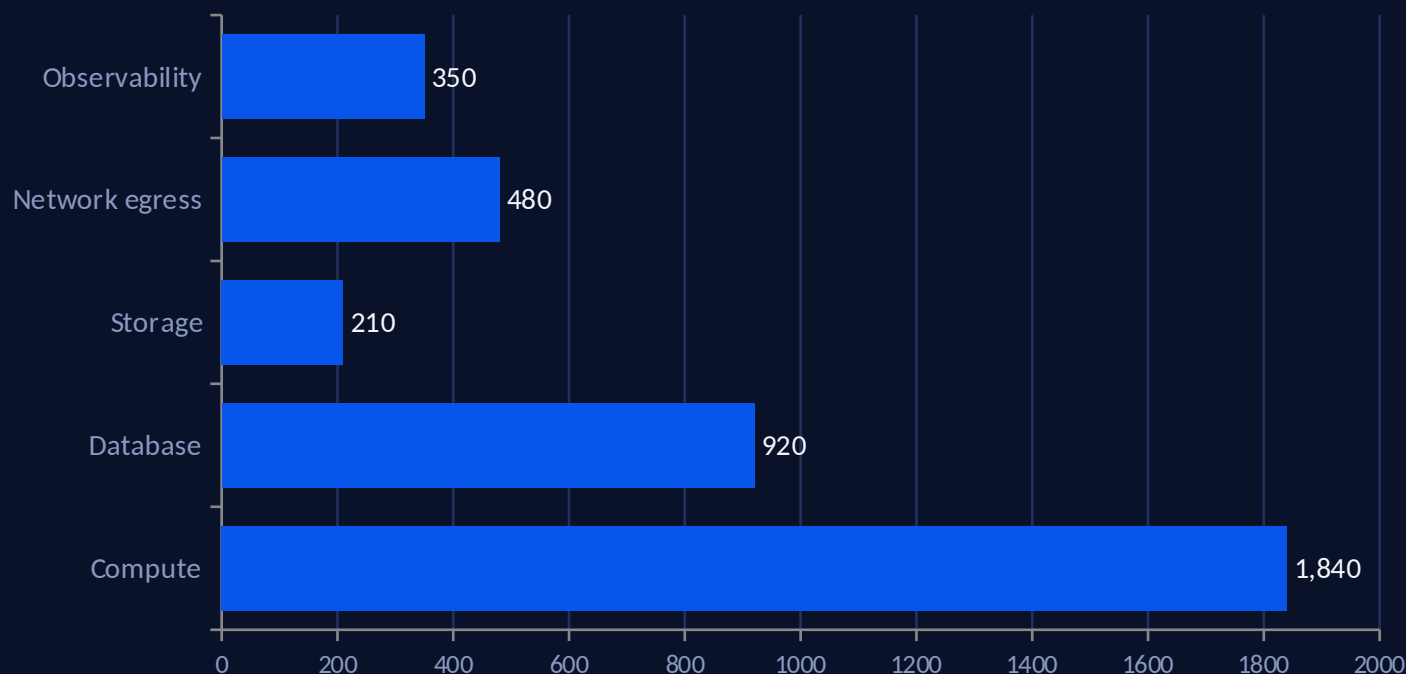
*Platform engineering*

The easy way and the right way are the same way. If not, that is your fault.



# The bill is a design document

*Illustrative figures for Daig - your architecture choices, priced.*



## Provision for peak, all day

~4x the compute bill, for capacity you use for twenty minutes.

## Scale on demand

Cheaper - but scale-up takes 90 seconds and the spike takes 20. You will drop orders.

**There is no right answer. There is a decision, and someone has to own it. That is the job.**

# Four questions about one request

Not four products. Four questions - and one request can answer all of them.

## Logs

**What happened?**

*The order failed at 18:43:12. Here is the stack trace.*

## Metrics

**How often, how bad, which way is it trending?**

*Error rate went from 0.1% to 6% in ninety seconds.*

## Traces

**Where in the chain did it go wrong?**

*Orders was fine. Dispatch took 8 seconds. It was waiting on the database.*

## Profiles

**Which line of code is burning the CPU?**

*One function. 71% of the time. Somebody wrote a loop inside a loop.*

**One standard covers all four now: OpenTelemetry. Learn it once, use it with any vendor.**

# AI is already in this pipeline

You have used it since first year. Let us be adults about it.

## Already normal

Code review, test generation, incident summaries, postmortem first drafts. Not novelty - Tuesday.

## Arriving now

Agents that diagnose and propose a fix on their own. The open question is who approves a machine's production change. Nobody has solved this yet - it might be you.

## Still yours

Judgement. Knowing which of four plausible answers is correct for your system, your budget, your users.

**The rule for this rotation: use any tool you like. You must be able to explain every line you ship.  
If you cannot explain it, you did not write it - and it does not go to production.**

# Hands up if you think this is true

DevOps means CI/CD

DevOps means we do not need an ops team

DevOps is about picking the right tools

Automate everything and the bottlenecks go away

We should do what Netflix does

All five

**FALSE**

*Ten seconds each. Then we move on.*

# Six days, one system

## DAY 1

**Git, Linux &  
Networking**

The ground everything  
stands on

## DAY 2

**Cloud &  
Terraform**

Infrastructure you can  
rebuild

## DAY 3

**Ansible &  
Docker**

Configure it, then  
containerise it

## DAY 4

**CI/CD &  
Observability**

Ship it, then see it

## DAY 5

**DevSecOps**

Break it before  
someone else does

## DAY 6

**Kubernetes &  
Demo Day**

Scale it, then defend it

Day 1 you fix one container by hand. Day 6, forty containers fix themselves.  
Everything in between is how automation replaced you doing it manually - and why that is a promotion.

# Day 1 - Git, Linux & Networking

*Before Daig runs anywhere, it runs on a machine - and that machine has to talk to other machines.*

## YOU WILL BUILD

- SSH into a fresh VM and find your way around
- Find the process eating the CPU. Kill it.
- Follow one request: DNS, then TCP, then HTTP
- Read ss / netstat output without panicking

## CHAOS HOUR - 16:00

Orders cannot reach the database. Firewall, wrong port, or DNS? Three teams, three different planted causes.

## SKIP THIS DAY AND

**80% of the "Kubernetes problems" you hit on Friday are Linux and networking problems wearing a costume.**

# Day 2 - Cloud & Infrastructure as Code

*Daig needs somewhere to live. Click it into existence once - then never again.*

## YOU WILL BUILD

- Provision Daig by hand in the console. Feel it.
- Destroy all of it.
- Rebuild the same thing in Terraform
- Change one variable. Read the plan before you apply.

## CHAOS HOUR - 16:00

Someone changed a resource in the console behind your back. State and reality disagree. Reconcile it.

## SKIP THIS DAY AND

**Infrastructure nobody can reproduce, nobody can review, and nobody can rebuild after somebody deletes it.**

# Wednesday - Config Management & Containers

*The box exists. Make it useful - and then make the box irrelevant.*

## YOU WILL BUILD

- Ansible playbook to configure the VM
- Run it twice. Notice what does not happen.
- Containerise a Daig service from scratch
- docker compose up the whole stack

## CHAOS HOUR - 16:00

Monday's crash loop is back - but now there are three possible causes and you have the vocabulary to name them.

## SKIP THIS DAY AND

**"Works on my machine", for the rest of your career.**



# Day 4 - CI/CD & Observability

*Ship it without touching it. Then see what you actually shipped.*

## YOU WILL BUILD

- A pipeline that tests, builds and deploys on push
- Break the build on purpose. Watch it refuse to deploy.
- Instrument a service with OpenTelemetry
- Find the slow endpoint in a trace. Find the hot function in a profile.

## CHAOS HOUR - 16:00

Daig is slow at iftar. Nobody tells you which service.  
Find it in the traces.

## SKIP THIS DAY AND

**You ship blind and you debug by guessing. Both get more expensive every year of your career.**

# Day 5 - DevSecOps

*Six real vulnerabilities are live in Daig right now. Find them before somebody else does.*

## YOU WILL BUILD

- Run the full gate chain: secrets, SAST, SCA, IaC, image, DAST
- Triage the findings - which actually matter here?
- Find the one every scanner missed
- Fix three, then write a gate so they cannot come back

## CHAOS HOUR - 16:00

Five of six get flagged automatically. One does not. Find it, and work out why no tool could have.

## SKIP THIS DAY AND

**You ship other people's vulnerabilities and find out from someone who is not on your side.**

# Day 6 - Kubernetes & Demo Day

*One container is easy. Forty that find each other, restart themselves and survive a node dying - that is orchestration.*

## YOU WILL BUILD

- Deploy Daig to a cluster: pods, deployments, services
- Scale it up. Then delete a pod and watch it return.
- Roll out an update. Then roll it back.
- Afternoon: Demo Day

## CHAOS HOUR - 16:00

Demo Day. I break your deployment in front of the room. You have ten minutes and a microphone.

## SKIP THIS DAY AND

**You can run software. You cannot run software at scale - which is most of what the job is.**

# Demo Day

Last afternoon. You own a Daig deployment. I break it in front of the room. Ten minutes and a microphone.

40%

**Diagnosis**

Did you read logs, metrics and traces - or restart things and hope?

30%

**The fix**

Does it work, and did you fix it at the right layer?

20%

**Explanation**

Can you explain it to someone non-technical? That is the senior skill.

10%

**Composure**

What you do when your first hypothesis turns out to be wrong.

Diagnosis outweighs the fix deliberately. A well-reasoned failure scores above a lucky guess.

# Scope aap main hai, field mein nahi.

Six days. A system that will genuinely break.  
And anyone here will answer any question you ask.

See you Day 1, 09:00. Bring a laptop with Docker on it.

# Before Monday morning

1

**Install Docker Desktop**

Then run: `docker run hello-world`. If that works, you are ready.

2

**Bring the laptop you will use all week**

With admin rights on it. This matters more than you think.

3

**No prep reading required**

Deliberately. Come with questions instead.

4

**Contact**

[adeel.arshad@tkxel.com](mailto:adeel.arshad@tkxel.com) - ask me anything, including on Sunday.